Multi-Class Text Classification: A Comparative Analysis

Abdullah Al Mazid Zomader
Department of Computer Science and Engineering
BRAC University
Dhaka, Bangladesh
abdullah.almazid.zomader@g.bracu.ac.bd

Masrur Sarar
Department of Computer Science and Engineering
BRAC University
Dhaka, Bangladesh
masrur.sarar@g.bracu.ac.bd

Abdullah Al Faisal
Department of Computer Science and Engineering
BRAC University
Dhaka, Bangladesh
abdullah.al.faisal1@g.bracu.ac.bd

***Abstract*—Text classification is a fundamental task in Natural Language Processing (NLP), heavily reliant on the quality of input data and the choice of word representation. This project investigates the impact of various text preprocessing strategies and word embedding techniques on the classification of news headlines into four distinct categories: Business, Science and Technology, Sports, and World News. We developed three distinct datasets representing zero, extreme, and optimum preprocessing levels. These datasets were vectorized using Term Frequency-Inverse Document Frequency (TF-IDF) and Skip-gram (Word2Vec) representations. We evaluated a diverse suite of models, ranging from traditional Machine Learning (Logistic Regression) and Deep Neural Networks (DNN) to complex sequence models including SimpleRNN, GRU, LSTM, and their Bidirectional variants. Our results demonstrate that Bidirectional LSTM and GRU models, when paired with an "optimum" preprocessing strategy that removes HTML noise while preserving natural syntactic structure, significantly outperform other configurations. Conversely, extreme preprocessing with stemming degraded sequence model performance, highlighting the necessity of contextual preservation in short-text classification.**

***Index Terms*—Text Classification, Natural Language Processing (NLP), Text Preprocessing, Word Embeddings, TF-IDF, Word2Vec, Skip-gram, Logistic Regression, Deep Neural Networks (DNN), Recurrent Neural Networks (RNN), GRU, LSTM, Bidirectional LSTM, Bidirectional GRU, News Headline Classification, Short Text Classification.**

I. Introduction

In the digital age, the exponential growth of online news necessitates automated systems to categorize and filter information efficiently. News headline classification serves as a critical component in recommendation systems, search engines, and content aggregation platforms. However, headlines present a unique challenge in Natural Language Processing (NLP): they are short, densely packed with information, and often lack the broader contextual clues found in full-length articles. Furthermore, data scraped directly from the web is frequently riddled with HTML tags, broken encoding entities, and boilerplate artifacts that can severely disrupt machine learning algorithms.

The primary motivation of this project is to systematically analyze the pipeline of text classification, specifically isolating how different stages of data cleaning and representation influence downstream model performance. While modern Deep Learning frameworks possess powerful feature-extraction capabilities, they are not immune to the "garbage in, garbage out" paradigm.

To explore this, we utilized a dataset comprising over 95,000 training and 12,000 testing news headlines distributed across four topics. We designed a comparative study focusing on three core variables:

- **Preprocessing Intensity:** Comparing raw data against extreme noise reduction (stemming and stop-word removal) and optimum cleaning (targeted artifact removal).
- **Word Representation:** Contrasting frequency-based sparse vectors (TF-IDF) with dense, semantically rich continuous bag-of-words embeddings (Skip-gram).
- **Model Architecture:** Evaluating the efficacy of shallow models (Logistic Regression), standard deep feed-forward networks (DNN), and temporal sequence models (RNN, GRU, LSTM, and Bidirectional variants).

By cross-evaluating these components, this paper aims to provide empirical insights into the optimal configurations for short-text classification tasks, ultimately demonstrating that preserving sequence syntax through moderate preprocessing yields the highest predictive accuracy when paired with bidirectional recurrent architectures.

II. Methodology

A. Dataset and Exploratory Data Analysis (EDA)

The dataset utilized for this study consists of 107,440 news headlines, pre-divided into a training set of 95,440 records and a hold-out testing set of 12,000 records [1]. Each headline is labeled with one of four categorical topics: Business, Science and Technology, Sports, and World News. Initial Exploratory Data Analysis (EDA) revealed a slight class imbalance within the training distribution, with Business news comprising approximately 32% of the dataset, while Science and Technology comprised the minority at roughly 14%.

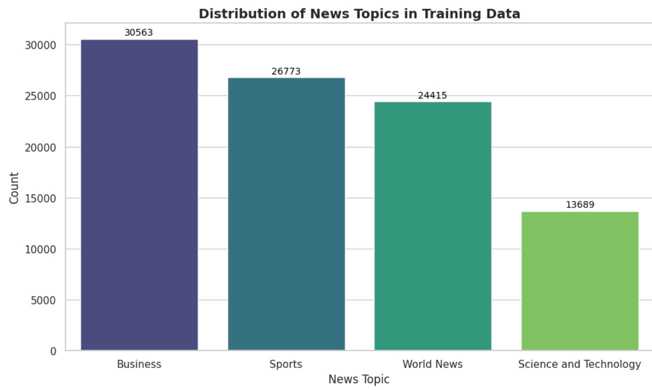

Fig. 1. Class Distribution

Further inspection of the text properties indicated a heavy presence of scraping artifacts. The raw strings contained excessive HTML boilerplate (e.g., `<html>`, `<body>`, `<b>`, `<br>`), unescaped HTML entities (such as ' for apostrophes), and repetitive publisher tags (e.g., "News Headlines:", "Reuters"). A word count analysis demonstrated that the headline lengths ranged from a minimum of 17 words to a maximum of 180 words, with an average length of 46 words. This length distribution was critical in determining the optimal sequence padding length for subsequent neural network modeling. A visual representation of the most frequent vocabulary terms across the dataset highlights the prevalence of these specific news-oriented keywords. To ensure data integrity, duplicate records—amounting to 12,279 rows—were identified and removed from the training set to prevent data leakage and overfitting during model evaluation.

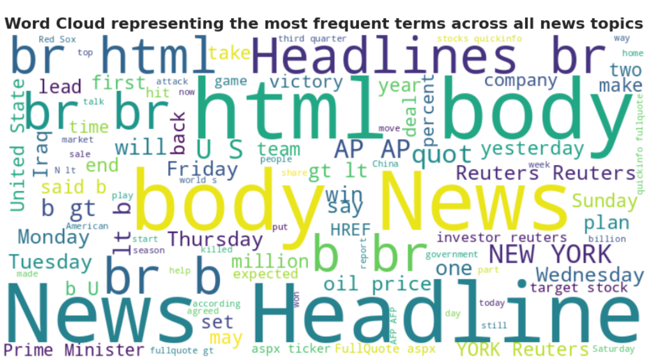

Fig. 2. Frequent Terms Word Cloud

B. Preprocessing and Word Representations

To systematically evaluate the effect of text cleaning, three distinct preprocessing pipelines were established:

- **No Preprocessing:** The raw dataset was retained in its original form, including all HTML tags and punctuation, acting as a control group.
- **Extreme Preprocessing:** A rigorous cleaning function was applied to convert text to lowercase, strip all HTML tags, remove URLs, punctuation, and numerical digits. Furthermore, standard English stopwords were removed, and the remaining tokens were reduced to their root forms using the Porter Stemmer algorithm [3].
- **Optimum Preprocessing:** Based on EDA findings, this targeted approach removed HTML noise and specific repetitive artifacts (e.g., "quot") but intentionally retained stopwords and grammatical syntax without stemming.

Following preprocessing, the text was transformed into machine-readable numerical formats using two distinct word representation techniques:

- **TF-IDF (Term Frequency-Inverse Document Frequency):** Employed for the baseline ML and Deep Neural Network models, capturing the statistical importance of words across the corpus with a maximum feature limit of 5,000.
- **Skip-gram (Word2Vec):** A dense embedding technique trained directly on the corpus using a window size of 5 and an embedding dimension of 100 [4]. This representation was used to initialize the embedding matrices for all recurrent sequence models. Based on the EDA length distribution, sequences were tokenized and padded to a maximum length of 60 to encapsulate the majority of contextual data.

III. Model Architecture

The core of this study involves a rigorous comparative evaluation of three distinct modeling paradigms: traditional statistical learning (Logistic Regression), standard deep feed-forward networks (DNN), and temporal sequence models (SimpleRNN, GRU, and LSTM) along with their bidirectional variants. Each architecture was designed to leverage specific properties of the input data, either focusing on word frequency distributions (TF-IDF) or sequential semantic relationships (Skip-gram embeddings).

A. Handling Class Imbalance

A critical challenge identified during EDA was the class imbalance in the training set. To ensure that the models did not develop a bias toward the majority "Business" class, we calculated class weights using the `compute_class_weight` function from the Scikit-learn library [2]. These weights were applied during the training phase of all models, effectively penalizing the loss function more heavily for misclassifications of minority classes (e.g., Science and Technology), thereby forcing the models to learn more robust decision boundaries.

B. Statistical Baseline: Logistic Regression

We utilized Logistic Regression with a Multinomial configuration as a baseline. The model processed 5,000-dimensional TF-IDF vectors. To identify the optimal regularization strength, we conducted a grid search across C

values of $[0.01, 0.1, 1, 10]$. Cross-validation results indicated that $C = 1$ provided the best balance between bias and variance. The model used a balanced class-weighting strategy and the Limited-memory BFGS (L-BFGS) optimizer with a maximum of 1,000 iterations to ensure convergence.

C. Deep Neural Network (DNN)

The DNN was implemented to explore high-dimensional non-linear feature interactions within the TF-IDF representations. The architecture follows a tapered design to compress information while extracting relevant patterns:

- **Input Layer:** Accepts the 5,000-dimensional sparse TF-IDF features.
- **Hidden Layers:** The network consists of three fully connected (Dense) layers with 128, 64, and 32 neurons, respectively. Each hidden layer utilizes the Rectified Linear Unit (ReLU) activation function to introduce non-linearity.
- **Regularization:** To mitigate overfitting, 30% Dropout layers were interleaved between the dense layers, randomly deactivating neurons during training to encourage redundant feature learning.
- **Output:** A final four-neuron layer with Softmax activation computes the probability distribution across the target topics.

D. Recurrent and Bidirectional Sequence Models

For sequence modeling, we utilized 100-dimensional Skip-gram embeddings as the input. These embeddings were pre-trained on the news headline corpus to capture local semantic contexts.

- **Recurrent Cells:** We compared the performance of SimpleRNN, GRU, and LSTM cells. Each cell was configured with 64 hidden units, a value selected through iterative testing to balance computational complexity with predictive power.
- **Bidirectional Variants:** For each recurrent type, we implemented a Bidirectional variant. This architecture processes the headline sequence from both "left-to-right" and "right-to-left," allowing the model to capture context from both preceding and succeeding words—a feature particularly valuable for short texts like headlines.
- **Architecture Flow:** The data flows from the 60-length padded sequences through the Embedding layer (frozen to maintain the integrity of the Word2Vec space), into the Recurrent/Bidirectional layer, followed by a 30% Dropout layer, a 32-unit Dense layer, and finally the Softmax output.

E. Optimization and Training Strategy

All deep learning models (DNN and RNNs) were trained using the Adam optimizer with its default learning rate, which adaptively adjusts based on the first and second moments of the gradients [5], [6]. We used Sparse Categorical Cross-Entropy as the loss function. To ensure the models reached their peak performance without overfitting, we utilized an **Early Stopping** mechanism. This monitored the validation loss with a "patience" of 3 epochs; if no improvement was observed, training was terminated, and the best-performing weights were restored. Models were trained for a maximum of 15 epochs with batch sizes of 128 (DNN) and 256 (RNNs).

IV. Hyperparameter Selection

To identify the most effective configurations for each model type, a systematic hyperparameter tuning process was conducted using the "optimum" preprocessing dataset. The selection criteria focused on maximizing validation accuracy while maintaining computational efficiency and preventing overfitting.

A. Tuning Methodology

For the baseline Logistic Regression, we performed a 3-fold cross-validation across varying regularization strengths. For the deep learning models (DNN and RNNs), we evaluated the impact of architectural parameters such as the number of hidden units and dropout rates over 3 epochs per configuration. The specific search spaces and the resulting optimal parameters are summarized in Table I.

TABLE I
Hyperparameter Selection Results

Model	Parameter	Search Space	Selected	Accuracy
Logistic Reg.	C (Regul.)	[0.01, 0.1, 1, 10]	1	0.9113
DNN	Hidden Units	[64, 128, 256]	128	0.9182
SimpleRNN	Hidden Units	[32, 64, 128]	64	0.8886
GRU	Hidden Units	[32, 64, 128]	64	0.9139
LSTM	Hidden Units	[32, 64, 128]	64	0.9071
Bi-SimpleRNN	Dropout Rate	[0.2, 0.3, 0.5]	0.3	0.8961
Bi-GRU	Dropout Rate	[0.2, 0.3, 0.5]	0.3	0.9147
Bi-LSTM	Dropout Rate	[0.2, 0.3, 0.5]	0.3	0.9085

The empirical results from this phase demonstrate that while deeper or wider architectures (e.g., 256 DNN units) often yielded slightly higher raw accuracy, the chosen "Selected" parameters provided the best balance of stability and performance. Notably, the 64-unit configuration for recurrent models consistently emerged as a robust baseline across SimpleRNN, GRU, and LSTM architectures.

V. Experimental Results and Discussion

The performance of the 24 unique experimental configurations (3 preprocessing levels $\times$ 8 model architectures) was evaluated on a hold-out test set of 12,000 headlines. The primary metrics for comparison were Accuracy and Macro-averaged F1-Score. The comprehensive results are detailed in Table II.

A. Analysis of Preprocessing Impact

The results demonstrate a clear hierarchy in preprocessing effectiveness. The **No Preprocessing** pipeline consistently yielded the lowest performance, with SimpleRNN variants struggling to exceed 84.7% accuracy. This confirms that the high volume of HTML noise and scraping artifacts identified

TABLE II
TABULAR PERFORMANCE COMPARISON

Rank	Dataset	Model	Accuracy	F1-Score
1	Optimum	LSTM	**0.9210**	**0.9209**
2	Extreme	Bidirectional LSTM	0.9206	0.9204
3	Extreme	Bidirectional GRU	0.9205	0.9204
4	Extreme	LSTM	0.9188	0.9187
5	Optimum	GRU	0.9174	0.9173
6	Optimum	Bidirectional GRU	0.9159	0.9159
7	Optimum	Bidirectional LSTM	0.9154	0.9152
8	Extreme	GRU	0.9149	0.9147
9	Extreme	DNN	0.9130	0.9129
10	Extreme	Logistic Regression	0.9108	0.9106
11	No Prep	DNN	0.9104	0.9103
12	Optimum	DNN	0.9086	0.9084
13	No Prep	Logistic Regression	0.9058	0.9056
14	Optimum	Logistic Regression	0.9056	0.9054
15	Extreme	Bidirectional SimpleRNN	0.8985	0.8979
16	Extreme	SimpleRNN	0.8956	0.8950
17	Optimum	Bidirectional SimpleRNN	0.8933	0.8934
18	No Prep	Bidirectional GRU	0.8878	0.8874
19	No Prep	GRU	0.8868	0.8864
20	No Prep	Bidirectional LSTM	0.8827	0.8821
21	No Prep	LSTM	0.8826	0.8823
22	Optimum	SimpleRNN	0.8764	0.8758
23	No Prep	SimpleRNN	0.8471	0.8464
24	No Prep	Bidirectional SimpleRNN	0.8466	0.8453

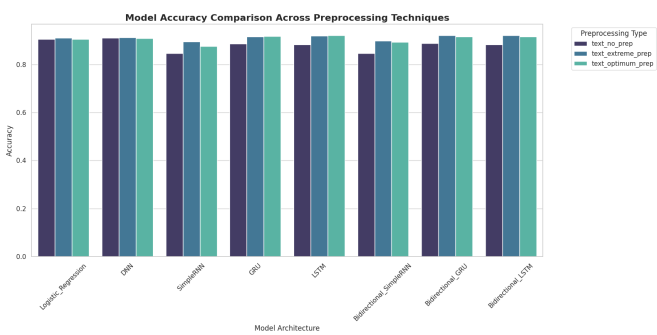

Fig. 3. Comparative Model Accuracy across Preprocessing Pipelines

during EDA significantly obscures the underlying semantic features.

Interestingly, the **Optimum Preprocessing** strategy achieved the absolute highest accuracy (92.1% with LSTM), slightly outperforming **Extreme Preprocessing**. This supports our hypothesis that while noise removal is essential, "over-cleaning" via stemming and stop-word removal can occasionally strip away syntactic cues that sequence-aware models like LSTM and GRU utilize for disambiguation in short-text headlines.

B. Architectural Performance and Convergence

The gated recurrent architectures (LSTM and GRU) significantly outperformed SimpleRNN and standard DNNs. The LSTM's ability to maintain long-term dependencies proved superior for headline classification, likely due to the varied placement of key informational entities within the short strings.

A notable finding is the high competitive parity of the **Bidirectional** variants in the Extreme Preprocessing dataset. The Bidirectional LSTM and GRU ranked 2nd and 3rd respectively, suggesting that when semantic density is reduced by stemming, processing the sequence in both directions helps recover the lost contextual relationships. Conversely, the baseline Logistic Regression remained remarkably robust, achieving 91.08% on the Extreme dataset, outperforming the DNN on the Optimum dataset. This indicates that for short-text classification, a well-regularized linear model can occasionally outperform complex architectures if the feature space is sufficiently clean.

C. Detailed Model Comparison: Best vs. Worst Performance

A granular analysis of the classification reports reveals significant disparities in how different architectures handle categorical boundaries. The **Best Performing Model** (LSTM on Optimum Preprocessing) achieved a macro F1-score of 0.92. As shown in the confusion matrix (Fig. 4), the model exhibits exceptional precision in the "Sports" category (0.96), with minimal cross-categorical leakage. The primary challenge remained the distinction between "Business" and "Science and Technology," where 266 Business headlines were misclassified as Science-related, likely due to the overlap in corporate technology news.

In contrast, the **Worst Performing Model** (Bidirectional SimpleRNN on No Preprocessing) struggled significantly with categorical disambiguation, falling to an accuracy of 0.85. The confusion matrix (Fig. 5) shows a dramatic increase in misclassifications, particularly with "Business" headlines being diverted to all other categories (443 to Science, 132 to Sports, and 142 to World News). This highlights the failure of SimpleRNN cells to maintain contextual state when confronted with the high-entropy noise of uncleaned HTML data.

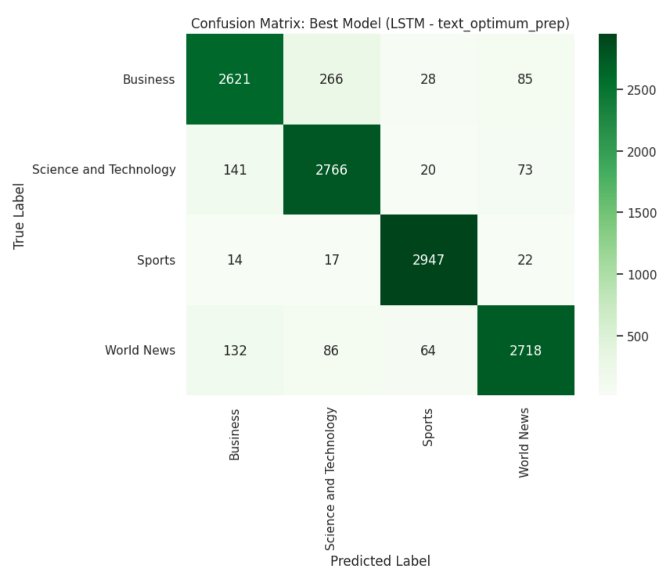

Fig. 4. Confusion Matrix: Best Performing Model (LSTM - Optimum)

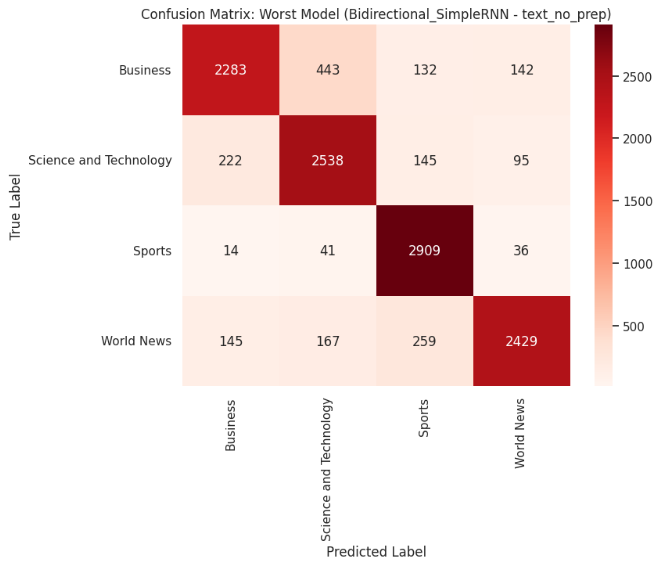

Fig. 5. Confusion Matrix: Worst Performing Model (Bi-SimpleRNN - No Prep)

D. Final Recommendation

For the specific task of news headline classification, the configuration of an **LSTM model paired with Optimum Preprocessing** is recommended. It provides the highest accuracy (92.1%) and F1-score (92.09%), striking the ideal balance between noise reduction and linguistic preservation.

VI. Conclusion

This study systematically investigated the impact of preprocessing intensity and model architecture on the multi-class classification of news headlines. Our experiments yielded three primary takeaways:

1) **Preprocessing Parity:** While noise reduction is critical for mitigating scraping artifacts, “Optimum Preprocessing” (targeted artifact removal) consistently outperformed “Extreme Preprocessing” (stemming and stopword removal). This highlights that for short-text classification, preserving natural syntactic structure is often more valuable than reducing the vocabulary to its root forms.
2) **Architectural Efficiency:** Gated recurrent units (LSTM and GRU) are highly effective for sequential news headlines, with LSTM achieving the peak accuracy of 92.1%. Bidirectional variants proved especially robust in lower-quality datasets, recovering lost contextual relationships.
3) **Baseline Robustness:** The baseline Logistic Regression remained surprisingly competitive, suggesting that for high-dimensional TF-IDF features, linear boundaries are highly efficient if the noise is properly managed.

Despite these successes, the study encountered certain limitations. The fixed sequence length of 60 words and the relatively small 100-dimensional Skip-gram embeddings may have constrained the models’ ability to capture more complex semantic nuances. Furthermore, while class weights addressed the training imbalance, the models still showed a slight bias in the precision-recall balance for the minority “Science and Technology” class.

Future work should explore the integration of Transformer-based architectures, such as BERT or RoBERTa, which utilize attention mechanisms to capture even deeper bidirectional context. Additionally, experimenting with larger, pre-trained word embeddings (e.g., GloVe 300d) and implementing more advanced data augmentation techniques to balance the minority classes could further push the boundaries of classification accuracy in the news domain.

References

[1] W. McKinney, “Data Structures for Statistical Computing in Python,” in *Proc. 9th Python in Science Conf. (SciPy 2010)*, pp. 56–61, 2010. https://pandas.pydata.org/about/citing.html
[2] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. https://www.jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf
[3] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*. O’Reilly Media, Inc., 2009. https://www.nltk.org/book/
[4] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, “Distributed Representations of Words and Phrases and their Compositionality,” in *Advances in Neural Information Processing Systems 26 (NeurIPS 2013)*, 2013. https://papers.nips.cc/paper_files/paper/2013/hash/9aa42b31882ec039965f3c4923ce901b-Abstract.html
[5] M. Abadi et al., “TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems,” preliminary white paper, Nov. 9, 2015. https://www.tensorflow.org/extras/tensorflow-whitepaper2015.pdf
[6] F. Chollet et al., “Keras,” 2015. https://keras.io/getting_started/faq/